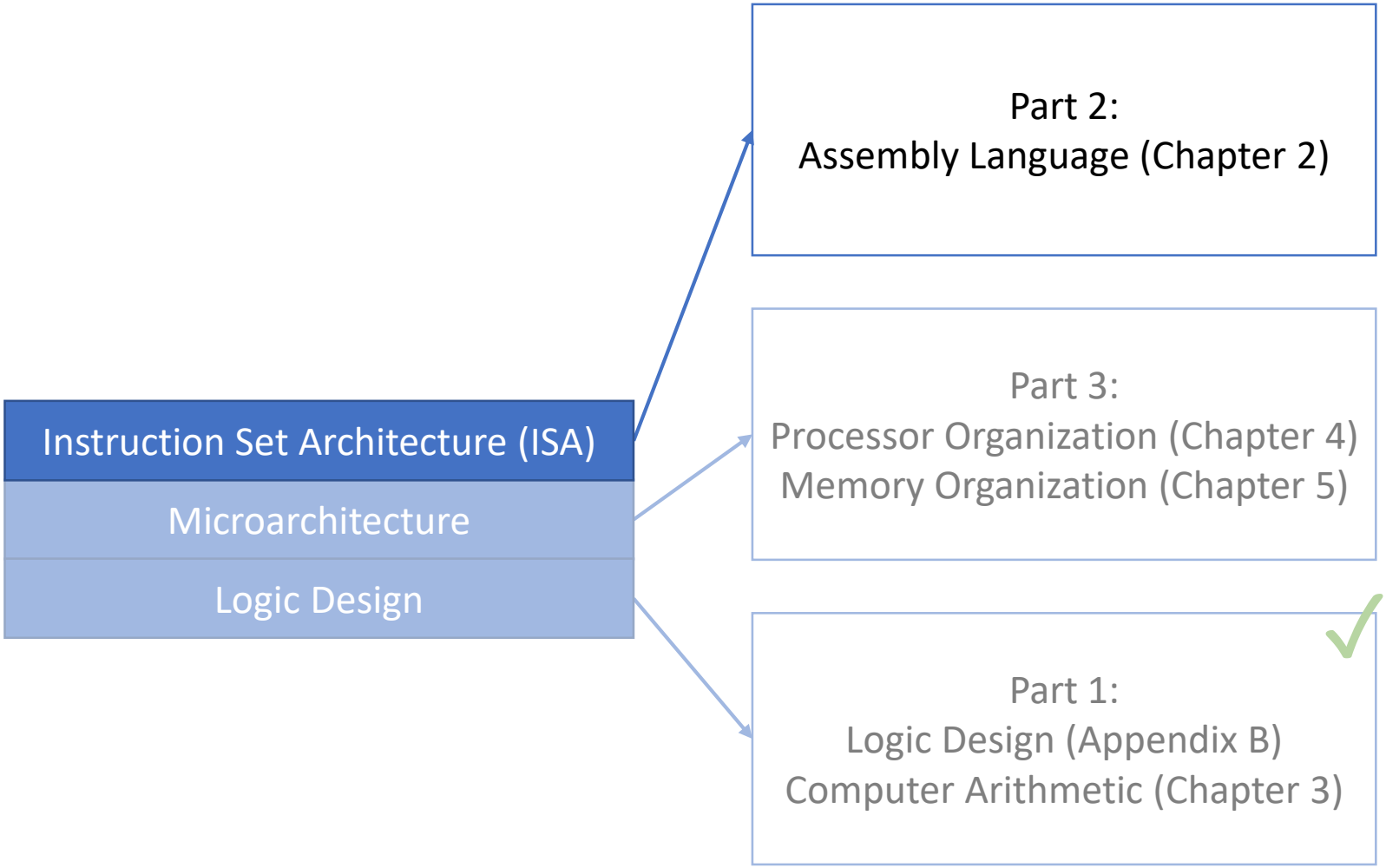
# Lecture 16: Memory Instructions

CMPS 221 – Computer Organization and Design

*Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.*

# Last Time: Arithmetic and Logic Instructions



# Instruction Set

- An **instruction set** is the repertoire of instructions recognized by a computer
- Different computers have different instruction sets
  - e.g., x86, ARM, Power, MIPS, RISC-V

# Arithmetic Instructions

- Add and subtract instructions
  - Three operands: two source, one destination

```
add a, b, c   # a = b + c
```

- All arithmetic operations have this form

# Register Operands

- Arithmetic instruction operands refer to registers stored in a register file
- MIPS has a 32 × 32-bit register file
  - A 32-bit unit of data is called a **word**
  - Registers numbered 0 to 31
- Registers are used for frequently accessed data
  - Less frequently accessed data is placed in memory

# Logical Operations

- Instructions for bitwise manipulation

| Operation | C | Java | MIPS |
|---|---|---|---|
| Bitwise AND | & | & | and, andi |
| Bitwise OR | \| | \| | or, ori |
| Bitwise NOT | ~ | ~ | nor |
| Shift left | << | << | sll |
| Shift right | >> | >>> | srl |

# Immediate Operands

- An **immediate operand** is a constant value specified in an instruction

```
addi $s3, $s3, 4
```

- No subtract immediate instruction
  - Just use a negative constant

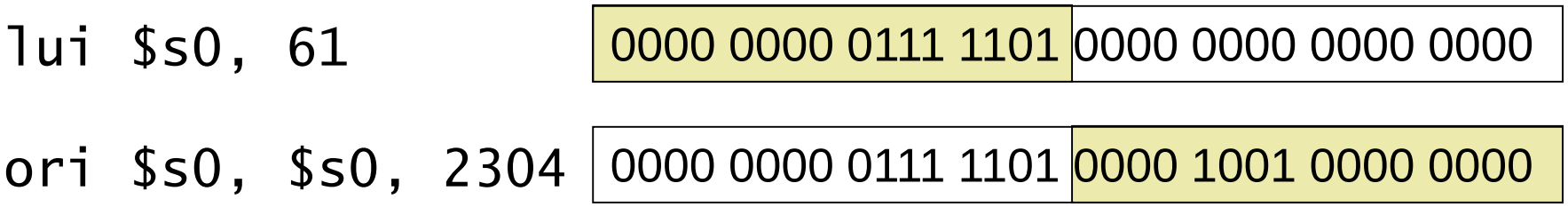
```
addi $s2, $s1, -1
```

- Limited to small constants up to 16 bits
  - We'll see why later

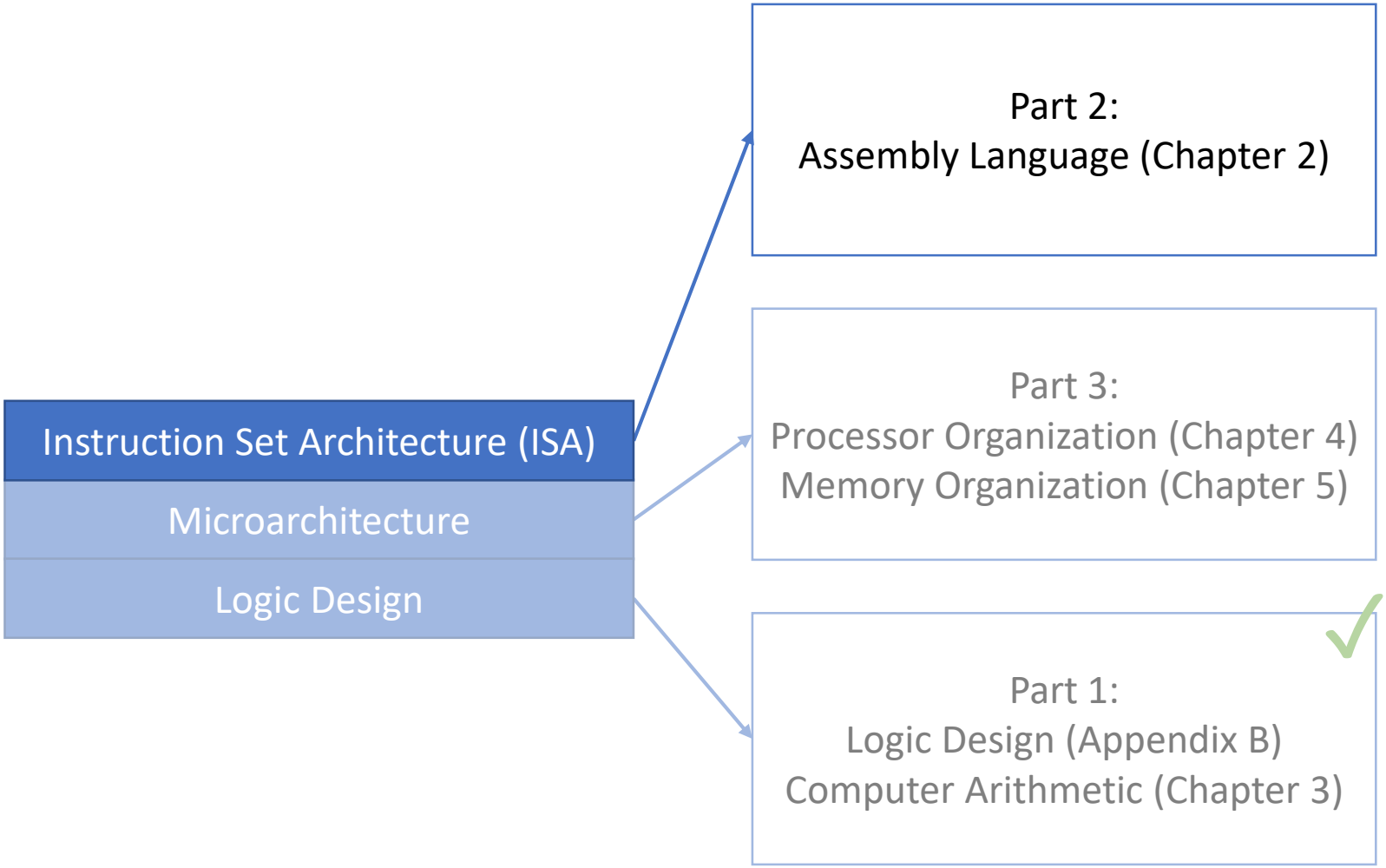
# 32-bit Constants

- Most constants are small
  - 16-bit immediate is sufficient
- For the occasional 32-bit constant

  `lui rt, constant`
  - Copies 16-bit constant to left 16 bits of rt
  - Clears right 16 bits of rt to 0
  - Combine with ori to create 32-bit constant



# Today: Memory Instructions

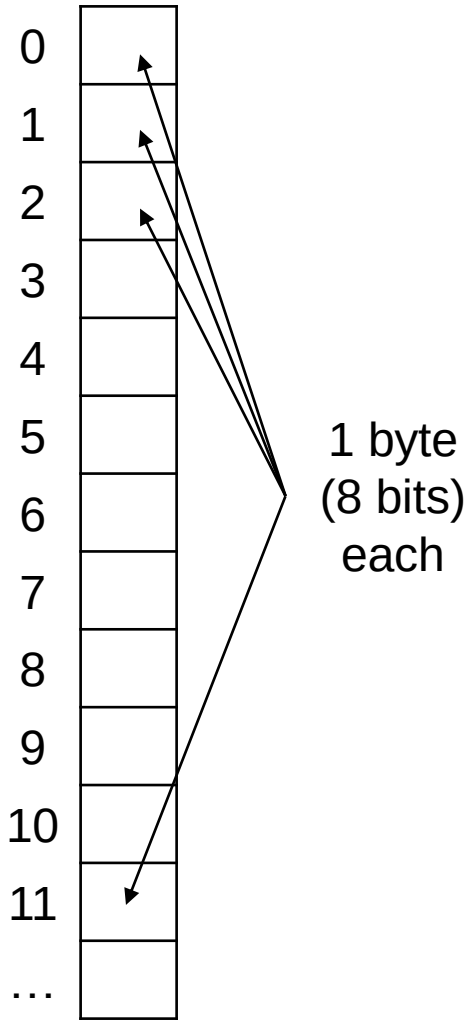


# Memory Instructions

- **Memory** is used for storing composite data
  - Arrays, structures, dynamic data
- To apply arithmetic operations to data in memory, MIPS requires separate instructions to:
  - **Load** values from memory into registers
  - **Store** result from register to memory
- Not all instruction sets have this requirement
  - Some instruction sets have instructions that apply operations to data in memory directly

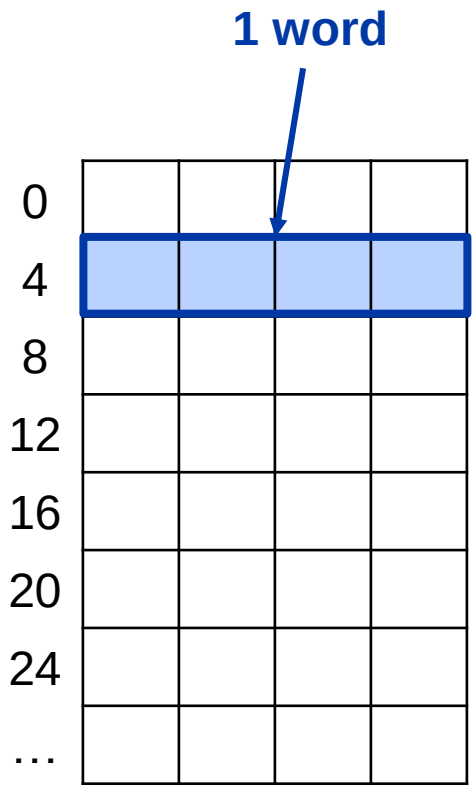
# Memory Addressing

- Memory is **byte-addressed**
  - Each address identifies an 8-bit byte



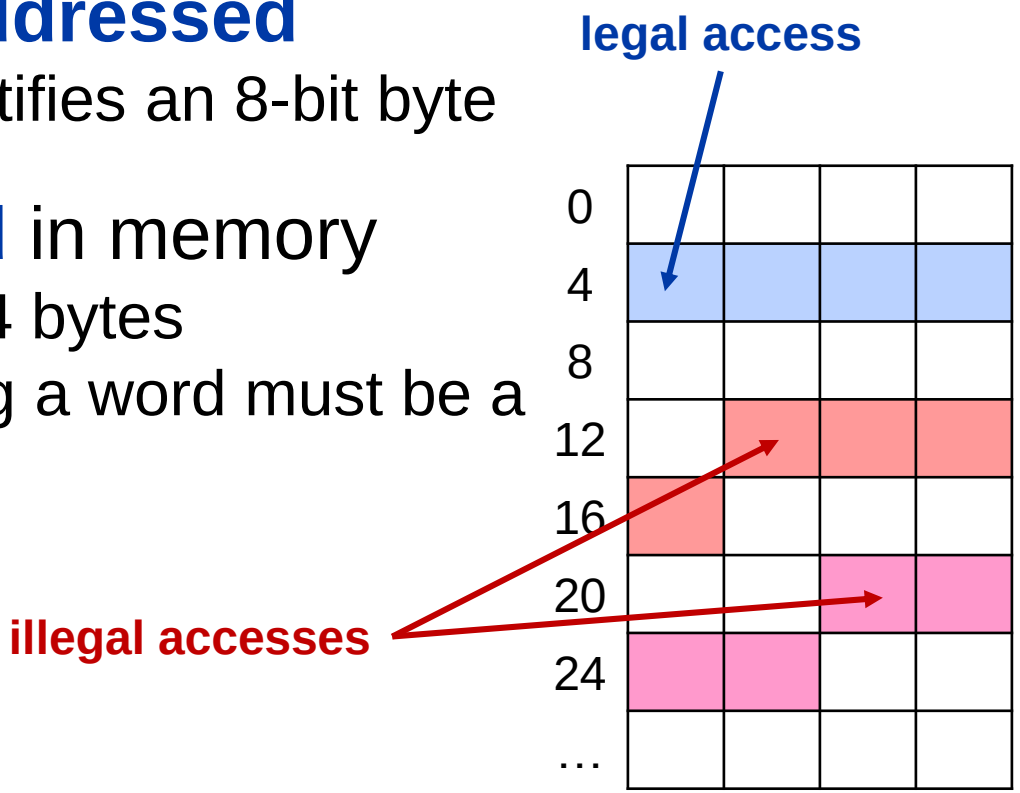
# Memory Addressing

- Memory is **byte-addressed**
  - Each address identifies an 8-bit byte
- Words are **aligned** in memory
  - Recall: words are 4 bytes



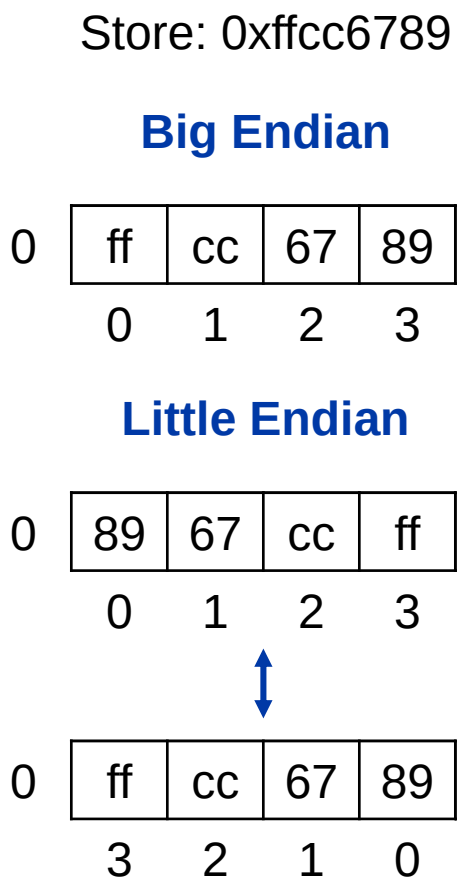
# Memory Addressing

- Memory is **byte-addressed**
  - Each address identifies an 8-bit byte
- Words are **aligned** in memory
  - Recall: words are 4 bytes
  - Address for loading a word must be a multiple of 4

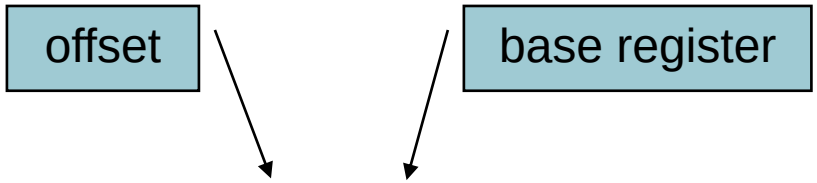


# Memory Addressing

- Memory is **byte-addressed**
  - Each address identifies an 8-bit byte
- Words are **aligned** in memory
  - Recall: words are 4 bytes
  - Address for loading a word must be a multiple of 4
- MIPS is **Big Endian**
  - Most-significant byte at least address of a word
  - In contrast, x86 is **Little Endian**: least-significant byte at least address



# Load/Store Word Instructions




```
lw $t0, 8($s0)
```

- Load a word from the address $s0+8 and place the result in $t0
- $s0+8 must be a multiple of 4 (alignment)
- Offset is limited to 16 bits (we'll see why later)

```
sw $s1, 12($t1)
```

- Store the word in $s1 at the address $t1+12
- $t1+12 must be a multiple of 4 (alignment)
- Offset is limited to 16 bits (we'll see why later)

# Memory Instructions Example

- C code:

```
A[5] = h + A[3];
```

  - h in $s2, base address of A in $s3
  - Assuming all integers

- Compiled MIPS code:
  - Index 3 requires offset of 12
    - 3 words x 4 bytes/word

```
lw  $t0, 12($s3)    # t0 = A[3]
add $t0, $s2, $t0
sw  $t0, 20($s3)    # A[5] = t0
```

# Memory Instructions Example

- C code:

```
A[i] = h + A[j];
```

  - i in $s0, j in $s1, h in $s2, base address of A in $s3
  - Assuming all integers

- Compiled MIPS code:

```
sll $t0, $s1, 2     # t0 = j*4
add $t0, $s3, $t0   # t0 = A + j*4
lw  $t0, 0($t0)     # t0 = A[j]
add $t0, $s2, $t0   # t0 = h + A[j]
sll $t1, $s0, 2     # t1 = i*4
add $t1, $s3, $t1   # t1 = A + i*4
sw  $t0, 0($t1)     # A[i] = t0
```

# Registers vs. Memory

- Registers are faster to access than memory
  - Operating on memory data requires loads and stores (more instructions to be executed)
- Use registers for variables as much as possible
  - Only put in memory less frequently used data

# Small Data

- Characters are smaller than words
  - ASCII: 128 characters
    - Requires 8 bits (1 byte) per character
  - Unicode: represents more characters
    - By default, uses 16 bits (1 halfword) per character
- Need a way to load/store data smaller than one word
  - Could use bitwise operations, but that requires multiple instructions (load, shift, clear)

# Byte/Halfword Instructions

`lb rt, offset(rs)  /  lh rt, offset(rs)`

- Sign extend to 32 bits in rt

`lbu rt, offset(rs) /  lhu rt, offset(rs)`

- Zero extend to 32 bits in rt

`sb rt, offset(rs)  /  sh rt, offset(rs)`

- Store just the rightmost byte/halfword

# Byte Access Example

- C code:

```
x[2] = x[0] + x[1];
```

  - x is an array of unsigned char whose address is in $s0

- MIPS code:

```
lbu $t0, 0($s0)   # Load x[0] to $t0
lbu $t1, 1($s0)   # Load x[1] to $t1
add $t0, $t0, $t1
sb  $t0, 2($s0)   # Store $t0 to x[2]
```

# Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 2.3, 2.9